Module 3

1. A simple exam scheduling CSP has subjectsAI*AI*,ML*ML*,DB*DB*with available slots{Monday,Tuesday,Wednesday}{*Monday*,*Tuesday*,*Wednesday*}. AI and ML cannot be scheduled on the same day, and ML and DB cannot be scheduled on the same day. If *ML* is scheduled on Tuesday and *AI* is scheduled on Monday, determine the possible day(s) for DB.

Ans: GivenML=Tuesday,AI=Monday*ML=Tuesday,AI=Monday*

The constrain isML≠DB. Therefore, DB cannot be scheduled on Tuesday.

The available days are Monday, Tuesday, and Wednesday.

Hence:DB∈{Monday,Wednesday}*DB*∈{*Monday*,*Wednesday*}

Both satisfy the given constraints.

2. Consider a CSP with variablesA,B,C*A*,*B*,*C*, where each variable has domain{1,2,3}{1,2,3}.The constraints are A≠B and B≠C. GivenA=1*A*=1,B=2*B*=2, Determine the possible values of C*C*.

Ans: GivenA=1,B=2*A*=1,*B*=2

First check A≠B, then1≠2

Hence, the first constraint is satisfied.

Now consider:B≠C SinceB=2,*C* cannot be 2. Therefore C∈{1,3}

(A,B,C)=(1,2,1).

Thus, the possible complete assignments are:

(A,B,C)=(1,2,1)

or

(A,B,C)=(1,2,3)

3. A CSP has variables X,Y,Z each with domain{1,2,3} and constraints X<Y and Y<Z. If *Y*=2, determine the possible values of *X* and *Z*.
   Ans: For,X<2,X=1
   For Z>2,Z=3
   Therefore (X,Y,Z)= (1,2,3)
4. A scheduling CSP has three activities A,B,C each of which can be scheduled in time slots{1,2,3}. The constraint is A≠B. If A=1 and B=2, determine whether the assignment is valid. Give a reason.
   Ans: The constraint isA≠B.
   Given A=1,B=2
   Since1≠2, the constraint is satisfied. Therefore, the assignment is **valid**.

5. A CSP has variables X, Y with domains $X, Y \in \{Red, Green, Blue\}$ and the constraint $X \neq Y$, $X = Green$. List the possible values for Y.

Ans: Since X=Green and $X \neq Y$ *Y*=cannot be Green So Y = {Red, Blue}

6. Apply the concept of Minimax to explain how a game-playing agent would choose a move when the opponent is assumed to play optimally.

Ans: The agent generates possible moves from the current game state and considers the opponent's possible responses. It evaluates the resulting game states using utility or evaluation values. The agent selects the move that gives the best outcome while assuming that the opponent will choose a response that is most unfavorable to the agent. Thus, Minimax helps the agent make decisions while considering optimal opponent behavior.

7. Discuss how alpha-beta pruning can reduce the number of nodes evaluated compared with Minimax. Use the condition for pruning to support your answer.

Ans: Minimax evaluates the game tree to determine the best move by considering the values of its branches. Alpha-beta pruning analyzes the same tree but avoids branches that cannot influence the final decision.

At a MAX node, alpha stores the best value currently available to MAX. At a MIN node, beta stores the best value currently available to MIN. When: $\alpha \geq \beta$

the remaining children of that node can be pruned because they cannot change the decision already obtained.

Thus, alpha-beta pruning produces the same optimal decision as Minimax while potentially evaluating fewer nodes. The amount of pruning depends on the order in which nodes are examined.

8. Explain how alpha-beta pruning improves the efficiency of Minimax search.

Ans: Alpha-beta pruning eliminates branches of the game tree that cannot affect the final decision. alpha represents the best value already guaranteed for MAX, while beta represents the best value already guaranteed for MIN. When alpha ≥ beta, further exploration of that branch is unnecessary.

9. Explain the role of MAX and MIN nodes in the Minimax algorithm.

Ans: In a game tree, **MAX** represents the player trying to maximize the utility value, while **MIN** represents the opponent trying to minimize it. At a MAX node, the highest value among the child nodes is selected. At a MIN node, the lowest value among the child nodes is selected.

$V(s) = \max_{a \in Actions(s)} V(Result(s,a))$

$V(s) = \min_{a \in Actions(s)} V(Result(s,a))$

10. Demonstrate why backtracking search may still require considerable search even though it rejects invalid assignments early.

Ans:
A CSP may contain many variables, each having several possible values, resulting in a large number of possible assignments. Although backtracking removes branches when constraints are violated, many partial assignments may remain valid for several levels of the search. If conflicts occur only after

several variables have been assigned, the algorithm may need to explore many branches before finding a solution.
11. Discuss the importance of checking constraints immediately after each assignment in backtracking search.

Ans: Immediate constraint checking allows the algorithm to detect invalid partial assignments as early as possible. Once a violation is detected, the corresponding branch can be abandoned without assigning values to the remaining variables. This prevents unnecessary exploration of invalid solutions. Therefore, early constraint checking can significantly reduce the number of assignments that need to be examined.

12. Explain the working of backtracking search with a suitable CSP example.

13.

Apply backtracking search to a map-coloring CSP in which adjacent regions cannot have the same color. Apply and discuss how the algorithm works when two adjacent regions receive the same color?

14.

Explain how backtracking search reduces unnecessary exploration of a CSP search space.

Ans: Backtracking checks constraints while constructing a solution rather than waiting until all variables are assigned. When a partial assignment violates a constraint, the corresponding branch is immediately abandoned. Thus, the algorithm does not generate further assignments from an already invalid partial solution.

15. Explain the main steps involved in backtracking search for a CSP.

1. Ans: Select an unassigned variable.
2. Select a value from its domain.
3. Check whether the assignment satisfies the relevant constraints.
4. If consistent, continue; otherwise, backtrack and try another value.

16.

Discuss the reason that backtracking does not continue with a partial assignment that violates a constraint.

Ans: A constraint-violating partial assignment cannot be extended into a valid solution while retaining that assignment. Continuing to explore it would therefore produce unnecessary search. Backtracking abandons such a branch and explores another possible assignment.

17. Demonstrate how backtracking handles a wrong assignment in a CSP.

Ans: Suppose X=Red is assigned a later assignment Y=Red violates the constraint X≠Y. The algorithm identifies the conflict, removes the conflicting assignment, and tries another value such asY=Blue

18. Discuss the scenario that happens when backtracking search encounters a constraint violation.

Ans: When a constraint violation occurs, the current assignment is considered invalid. The algorithm removes the most recent assignment and returns to the previous variable. It then tries another available value.

19.

Explain the terms variable, domain, and constraint in a CSP with an example.

20.

Discuss whether backtracking search is suitable for solving CSPs or not.

21.

Explain the basic idea of backtracking search in solving a Constraint Satisfaction Problem (CSP).

22.

A tic-tac-toe AI has two possible moves, A and B. Move A allows the opponent to create two immediate winning possibilities, while move B allows only one such possibility. Apply adversarial-search reasoning to determine which move is preferable and justify your answer.

Ans: Move **B** is preferable because it gives the opponent fewer opportunities to create a winning outcome. In adversarial search, the AI must consider the opponent's possible responses rather than evaluating only its immediate move. Since Move A provides two immediate winning possibilities for the opponent, it exposes the AI to greater risk. Move B reduces the opponent's potential advantage and is therefore the better defensive choice.

23.

Explain how adversarial search models decision-making when two players have conflicting goals.

Ans: Adversarial search represents a problem as a game involving multiple players. Each player selects actions based on their own objective, while anticipating that the opponent will try to achieve a conflicting objective. The search explores possible game states resulting from the players' actions and evaluates the outcomes to support decision-making

24. Discuss the main components of a game-playing problem in adversarial search.

25.

Discuss how adversarial search differs from ordinary search.

26.

A university has three examinations A, B, C and three time slots T1, T2, T3. Examinations A and B have common students while B and C have common students. Represent this as a CSP and give one valid assignment.

27.

Explain the role of a constraint graph in representing a CSP.

28.

Explain the difference between a constraint and a domain in a CSP using an example.

29.

Explain how a map-coloring problem can be represented as a CSP.

30.

Distinguish between unary and binary constraints with suitable examples.

31.

Consider variables X= {A,B}, with D(A) ={1,2} and D(B)= {2,3}. If the constraint is A<B, list all valid assignments.

32.

Differentiate between a variable and a domain in a CSP with a suitable example.

33.

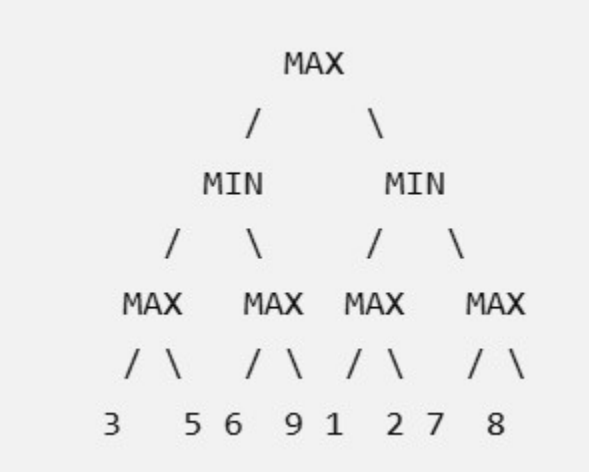
Define a constraint in a Constraint Satisfaction Problem with suitable example.

34.

Define a Constraint Satisfaction Problem (CSP). Identify its three basic components.

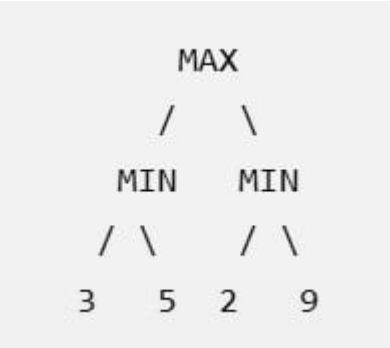
35.

Given the following game tree, apply Alpha-Beta Pruning from left to right and identify the pruned nodes.



36.

Consider the following game tree. The root is a MAX node and its two children are MIN nodes. The leaf values are evaluated from left to right. Determine where Alpha-Beta pruning occurs.



37.

Explain the working of Alpha-Beta Pruning with the roles of MAX, MIN, alpha, and beta.

38.

Consider a MAX node with two MIN children. The first MIN child returns a value of 5. While evaluating the second MIN child, its first child has value 3. Determine whether pruning can occur and justify your answer.

39.

Explain how Alpha-Beta Pruning improves the Minimax algorithm.

40.

Determine the optimal action if a MAX node has successors A, B and C with values 4, -1, 4.

41.

A game position has three available actions with minimax values 2, 0, 5. Demonstrate the MAX function here with a diagram and its notation.

42.

A MIN root has three MAX children with leaves (2,6),(4,5),(1,8). Illustrate and detail using suitable diagrams.